

HANDOUT THỰC HÀNH

End-to-End ML Workflow

Dự án thực tế: California Housing Price Prediction

Machine Learning in Data Science

Họ và tên	
MSSV	
Lớp	
Ngày thực hành	

Mục tiêu handout

Sau khi hoàn thành 3 phần thực hành này, sinh viên có thể độc lập xây dựng một pipeline ML hoàn chỉnh từ đầu đến cuối: từ phân tích dữ liệu, xây dựng pipeline tiền xử lý, huấn luyện và fine-tune mô hình, cho đến đánh giá và triển khai.

Tài nguyên cần chuẩn bị:

- Jupyter Notebook: 02_end_to_end_machine_learning_project.ipynb
- GitHub: <https://github.com/ageron/handson-mlp>
- Thư viện: scikit-learn, pandas, matplotlib, numpy, joblib

HANDOUT 1 — Khám phá dữ liệu & Tiền xử lý

Bước 1–4: Problem Framing → Data Collection → EDA → Preprocessing

PHẦN LÝ THUYẾT NỀN**1.1 Problem Framing (Bước 1)**

Trước khi viết bất kỳ dòng code nào, ta phải trả lời 3 câu hỏi cốt lõi:

Câu hỏi	Câu trả lời cho bài toán nhà đất
Loại bài toán ML?	Supervised Learning + Regression (dự đoán giá trị liên tục)
Đơn/đa biến?	Multiple Regression — dùng nhiều features (income, rooms, location...)
Phương pháp học?	Batch Learning — dữ liệu census 1990, không cần cập nhật liên tục

Performance Measure

RMSE (Root Mean Squared Error) — phổ biến nhất cho regression. Penalizes các lỗi lớn nặng hơn: $RMSE = \sqrt{\text{mean}((y_{\text{pred}} - y_{\text{true}})^2)}$. Ngoài ra có MAE (Mean Absolute Error) — ít bị ảnh hưởng bởi outliers hơn.

1.2 Data Collection & Train/Test Split (Bước 2)

Hai chiến lược split chính:

Tiêu chí	Random Sampling	Stratified Sampling
Khi dùng	Dataset lớn, phân phối đều	Dataset có biến quan trọng cần giữ tỷ lệ
Ưu điểm	Đơn giản, nhanh	Test set đại diện hơn cho population
Trong dự án	<code>train_test_split(random_state=42)</code>	<code>train_test_split(stratify=income_cat)</code>
Kết quả	16512 train / 4128 test	Giữ đúng tỷ lệ <code>income_cat</code> trong cả 2 set

1.3 EDA & Visualization (Bước 3)

Quy trình EDA chuẩn — 5 bước:

1. `housing.info()` — kiểm tra kiểu dữ liệu, missing values
2. `housing.describe()` — thống kê mô tả (mean, std, min, max, percentiles)
3. `housing.hist()` — phát hiện skew, capped values, scale khác nhau
4. Scatter plot địa lý — xác định clusters giá nhà theo location
5. Correlation matrix — chọn features quan trọng, tạo feature mới

Kết quả EDA quan trọng nhất trong dự án này

- median_income có correlation cao nhất với giá nhà ($r = 0.69$)
- Tạo feature mới: rooms_per_house, bedrooms_ratio cho correlation tốt hơn
- total_bedrooms có 207 missing values cần xử lý
- Giá nhà bị capped tại \$500,000 (có thể gây bias khi predict)

1.4 Data Preprocessing (Bước 4)

Scikit-learn Pipeline — nguyên tắc quan trọng nhất:

- fit() CHỈ gọi trên training data — tránh data leakage
- transform() áp dụng GIỐNG NHAU cho train và test
- Pipeline đảm bảo thứ tự đúng và tái sử dụng được

Các bước preprocessing trong dự án:

Vấn đề	Giải pháp	Scikit-learn API
Missing values	Median Imputation	<code>SimpleImputer(strategy='median')</code>
Categorical features	One-Hot Encoding	<code>OneHotEncoder(handle_unknown='ignore')</code>
Scale khác nhau	Standardization	<code>StandardScaler()</code>
Custom logic	FunctionTransformer	<code>FunctionTransformer(np.log1p)</code>

PHẦN THỰC HÀNH

Bài tập 1: Phân tích dataset

- ☐ Chạy `housing.info()` — ghi lại: bao nhiêu rows, bao nhiêu features, kiểu dữ liệu, missing values?
- ☐ Chạy `housing.describe()` — feature nào có khoảng giá trị rộng nhất?
- ☐ Vẽ histogram cho tất cả features — nhận xét: feature nào bị skewed? Feature nào bị capped?
- ☐ Tính `value_counts()` của `ocean_proximity` — có bao nhiêu category?

 Quan sát từ `housing.info()` và `describe()`:

Bài tập 2: Stratified Split

- ☐ Tạo cột `income_cat = pd.cut(median_income, bins=[0,1.5,3,4.5,6,np.inf], labels=[1,2,3,4,5])`
- ☐ Dùng `train_test_split` với `stratify=income_cat` để split 80/20

- ☐ So sánh tỷ lệ income_cat giữa train, test và toàn bộ dataset — chúng có giống nhau không?
- ☐ Sau khi split xong: drop cột income_cat và đặt housing = strat_train_set.copy()

Toàn bộ dataset (%)	Train set (%)	Test set (%)
Cat 1: _____		
Cat 2: _____		
Cat 3: _____		
Cat 4: _____		
Cat 5: _____		

Bài tập 3: EDA — Visualization & Correlation

- ☐ Vẽ scatter plot latitude vs longitude với s=population/100, c=median_house_value, cmap='jet'
- ☐ Tính correlation matrix:
housing.corr(numeric_only=True)['median_house_value'].sort_values(ascending=False)
- ☐ Tạo 3 features mới: rooms_per_house, bedrooms_ratio, people_per_house
- ☐ Tính lại correlation — các feature mới có cải thiện correlation không?
- ☐ Dùng scatter_matrix() để vẽ top 4 features tương quan cao nhất

Feature	Correlation với median_house_value
median_income	
rooms_per_house	
bedrooms_ratio	
people_per_house	
latitude	

Bài tập 4: Xây dựng Preprocessing Pipeline

- ☐ Tách X_train = strat_train_set.drop('median_house_value', axis=1) và y_train
- ☐ Tạo num_pipeline: SimpleImputer → StandardScaler
- ☐ Tạo cat_pipeline: SimpleImputer(strategy='most_frequent') → OneHotEncoder
- ☐ Kết hợp bằng ColumnTransformer hoặc make_column_transformer()
- ☐ Gọi preprocessing.fit_transform(X_train) và kiểm tra shape output

Code pipeline của bạn (viết tay sketch):

 Sketch pipeline code tại đây:

Shape của X_train sau khi transform: _____ × _____

Giải thích: tại sao số cột tăng lên sau khi preprocessing?

 Giải thích:

Câu hỏi phản tư (Self-reflection)

1. Tại sao phải gọi fit() chỉ trên training data? Điều gì xảy ra nếu fit() trên toàn bộ dataset?
2. Stratified sampling quan trọng hơn random sampling trong trường hợp nào?
3. Khi EDA, bạn phát hiện feature median_income bị skewed. Bạn sẽ xử lý nó như thế nào?

HANDOUT 2 — Model Selection, Training & Fine-Tuning

Bước 5–6: Model Training → Cross-Validation → Hyperparameter Tuning

PHẦN LÝ THUYẾT NỀN**2.1 Model Selection & Training (Bước 5)**

Ba mô hình được thử trong dự án, theo thứ tự độ phức tạp tăng dần:

Tiêu chí	Linear Regression	Decision Tree	Random Forest
Train RMSE	~\$68,232	~\$0 (overfit!)	Tốt nhất
CV RMSE	~\$68,000	~\$66,000	~\$17,500
Vấn đề	Underfitting	Severe overfitting	Promising ✓
Kết luận	Mô hình quá đơn giản	Học thuộc lòng training data	Chọn để fine-tune

2.2 Cross-Validation — Tại sao cần thiết?

Vấn đề với evaluate trực tiếp trên training set:

- Decision Tree RMSE = \$0 trên train set — nhưng thực ra đang overfit nặng
- Không phản ánh performance thực trên unseen data

K-Fold Cross-Validation giải quyết vấn đề này:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(tree_reg, X_train_prepared,
                          y_train, scoring='neg_mean_squared_error', cv=10)
rmse_scores = np.sqrt(-scores) # neg vì sklearn dùng convention tối thiểu hóa
print(rmse_scores.mean(), rmse_scores.std())
```

Lưu ý về scoring

Sklearn dùng convention maximize score, nên các loss metrics (MSE, RMSE) được đổi dấu thành `neg_mean_squared_error`. Kết quả trả về là giá trị âm — phải nhân với -1 rồi mới `sqrt()` để ra RMSE.

2.3 Hyperparameter Tuning (Bước 6)

Hai chiến lược chính:

Tiêu chí	GridSearchCV	RandomizedSearchCV
Cách hoạt động	Thử TẤT CẢ tổ hợp	Thử K tổ hợp ngẫu nhiên
Khi dùng	Search space nhỏ	Search space lớn
Ưu điểm	Đảm bảo tìm best	Nhanh hơn nhiều
Nhược điểm	Chậm khi space lớn	Không đảm bảo optimal
Ví dụ trong dự án	3×4 = 12 tổ hợp × 5-fold	n_iter=10 × 5-fold

PHẦN THỰC HÀNH

Bài tập 5: Huấn luyện và so sánh 3 mô hình

- ☐ Tạo full pipeline: preprocessing + LinearRegression → fit → tính RMSE trên train set
- ☐ Lặp lại với DecisionTreeRegressor — RMSE trên train set bằng bao nhiêu? Có bất thường không?
- ☐ Lặp lại với RandomForestRegressor(n_estimators=100) — RMSE trên train set
- ☐ Dùng cross_val_score(cv=10) cho tất cả 3 mô hình — ghi mean ± std RMSE
- ☐ Kết luận: mô hình nào promising nhất? Tại sao?

Mô hình	Train RMSE (\$)	CV Mean RMSE (\$)	CV Std (\$)
Linear Regression			
Decision Tree			
Random Forest			

Mô hình promising nhất: _____ — Lý do:



Bài tập 6: GridSearchCV cho Random Forest

- ☐ Thiết lập param_grid: thử n_estimators=[100,200,300] và max_features=[2,4,6,8]
- ☐ Chạy GridSearchCV(cv=5, scoring='neg_mean_squared_error')
- ☐ In ra grid_search.best_params_ và grid_search.best_score_
- ☐ Xem feature importances: grid_search.best_estimator_.feature_importances_
- ☐ Liệt kê top 5 features quan trọng nhất

Best params từ GridSearchCV:

Hyperparameter	Best value
n_estimators	
max_features	
Best CV RMSE (\$)	

Top 5 feature importances (tên feature + giá trị):

1. _____

2.	
3.	
4.	
5.	

Bài tập 7: RandomizedSearchCV — mở rộng search space

- ☐ Dùng scipy.stats distributions: randint(100, 500) cho n_estimators, randint(1, 10) cho max_features
- ☐ Thêm bootstrap=[True, False] vào param_distributions
- ☐ Chạy RandomizedSearchCV(n_iter=20, cv=5) — so sánh thời gian với GridSearch
- ☐ So sánh best_score_ giữa GridSearch và RandomizedSearch — cái nào tốt hơn?

Phương pháp	Best CV RMSE (\$)	Thời gian chạy
GridSearchCV		
RandomizedSearchCV		

Câu hỏi phản tư (Self-reflection)

- Decision Tree RMSE = 0 trên train set có nghĩa là gì? Đây có phải là mô hình tốt không?
- Nếu GridSearchCV best params ở biên của param_grid (ví dụ n_estimators=300 — giá trị max), bạn nên làm gì tiếp theo?
- Feature importance từ Random Forest có thể dùng để làm gì trong bước tiếp theo?

HANDOUT 3 — Evaluation, Deployment & Tổng kết

Bước 7–8: Final Evaluation → Model Deployment → End-to-End Review

PHẦN LÝ THUYẾT NỀN**3.1 Final Evaluation trên Test Set (Bước 7)****Nguyên tắc vàng — PHẢI nhớ**

Test set chỉ được mở ra ĐÚNG MỘT LẦN duy nhất, sau khi đã hoàn tất hoàn toàn việc chọn mô hình và tuning. Bất kỳ quyết định nào dựa vào test set đều làm contaminate nó — kết quả không còn đại diện cho real-world performance.

Quy trình evaluate đúng chuẩn:

6. Lấy `X_test` và `y_test` từ `strat_test_set`
7. Gọi `final_model.transform(X_test)` — KHÔNG được `fit()` trên test data
8. Tính RMSE trên `X_test_prepared` và `y_test`
9. Báo cáo con số này là final performance

Confidence Interval cho kết quả:

```
from scipy import stats

confidence = 0.95
squared_errors = (final_predictions - y_test) ** 2
interval = np.sqrt(stats.t.interval(confidence,
    len(squared_errors)-1,
    loc=squared_errors.mean(), scale=stats.sem(squared_errors)))
```

3.2 Model Deployment (Bước 8)

Pipeline trong Scikit-learn có thể save và load toàn bộ — bao gồm cả preprocessing steps:

Save model	Load & predict
<pre>import joblib joblib.dump(final_model, 'housing_model.pkl')</pre>	<pre>model = joblib.load('housing_model.pkl') preds = model.predict(X_new)</pre>

Lợi ích của Pipeline khi deploy

Khi save cả Pipeline (gồm preprocessing + model), bạn không cần apply từng bước thủ công khi predict. Chỉ cần load file .pkl và gọi `model.predict(X_raw)` — pipeline tự động xử lý imputation, encoding, scaling trước khi predict.

3.3 Key Takeaways — Workflow Tổng quát

1	Frame First: Xác định business objective, loại task, performance metric TRƯỚC khi chạm vào data
2	EDA trước: Hiểu data trước khi model. Histogram, scatter, correlation tiết lộ issues quan trọng

3	Pipeline chống leakage: fit() CHỈ trên train data; transform() áp dụng đồng nhất cho train và test
4	Cross-Validation: Luôn dùng CV để estimate real performance, không evaluate trên train set
5	Test set một lần: Mở test set đúng một lần, sau khi mọi quyết định đã được đưa ra

PHẦN THỰC HÀNH

Bài tập 8: Final Evaluation trên Test Set

- ☐ Lấy `X_test = strat_test_set.drop('median_house_value', axis=1)` và `y_test`
- ☐ Dùng `final_model` (best model từ `GridSearch` hoặc `RandomizedSearch`) để predict
- ☐ Tính RMSE cuối cùng trên test set — so sánh với CV RMSE ở Handout 2
- ☐ Tính 95% confidence interval cho test RMSE bằng `scipy.stats.t.interval()`
- ☐ Viết nhận xét: model có generalize tốt không? CV RMSE và Test RMSE chênh lệch bao nhiêu?

Chỉ số	Giá trị (\$)
Best CV RMSE (từ HO2)	
Final Test RMSE	
95% CI Lower Bound	
95% CI Upper Bound	
Chênh lệch CV vs Test RMSE	

Nhận xét về generalization của model:



Bài tập 9: Model Deployment & Testing

- ☐ Save best model: `joblib.dump(final_model, 'my_california_housing_model.pkl')`
- ☐ Load lại trong một cell mới: `loaded_model = joblib.load('my_california_housing_model.pkl')`
- ☐ Tạo `X_new` với 3 districts giả định (chọn giá trị features hợp lý)
- ☐ Chạy `loaded_model.predict(X_new)` — ghi lại predicted prices

☐ Thử nghiệm BONUS: thay đổi median_income của một district — giá nhà thay đổi như thế nào?

District	Đặc điểm chính	Predicted Price (\$)
District A (income cao, gần biển)		
District B (income trung bình, inland)		
District C (income thấp, xa biển)		

Bài tập 10: End-to-End Review — Tự đánh giá toàn bộ pipeline

- ☐ Vẽ sơ đồ toàn bộ 8 bước ML workflow từ đầu đến cuối (có thể dùng mũi tên)
- ☐ Với mỗi bước, ghi: input là gì → output là gì → công cụ Scikit-learn dùng
- ☐ Xác định: nếu train RMSE << CV RMSE, vấn đề là gì? Cách fix?
- ☐ Xác định: nếu CV RMSE ≈ Test RMSE, điều đó nói lên điều gì về model?
- ☐ BONUS: Thêm một feature mới vào pipeline — so sánh RMSE trước và sau

Sơ đồ 8 bước ML Workflow (sketch tại đây):

Bước	Input → Output	Sklearn API chính
1. Problem Framing	Business goal → ML task type	—
2. Data Collection	Raw data → Train/Test sets	<code>train_test_split()</code>
3. EDA	Train set → Insights, new features	<code>corr()</code> , <code>hist()</code> , <code>scatter()</code>
4. Preprocessing	Raw features → Scaled/encoded	Pipeline, ColumnTransformer
5. Model Training	Prepared data → Trained models	<code>fit()</code> , <code>cross_val_score()</code>
6. Fine-Tuning	Trained model → Best params	GridSearchCV, RandomizedSearchCV
7. Evaluation	Best model → Final RMSE	<code>predict()</code> , RMSE on test set
8. Deployment	Final model → Production file	<code>joblib.dump()</code> , <code>joblib.load()</code>

🔍 Câu hỏi phản tư (Self-reflection)

- Test RMSE (~\$42,123) cao hơn CV RMSE (~\$17,500) cho Random Forest. Điều này có nghĩa là gì? Đây có phải là kết quả xấu không?
- Nếu bạn cần deploy model này trong thực tế, bạn sẽ monitor những metrics nào theo thời gian?

3. Nếu sau 6 tháng deploy, RMSE trên production tăng mạnh, nguyên nhân có thể là gì?
Cách khắc phục?

TỔNG KẾT & TỰ ĐÁNH GIÁ

Đánh dấu vào những kỹ năng bạn đã thực sự hiểu và làm được:

- ☐ Phân tích và giải thích output của `info()`, `describe()`, `hist()` cho dataset mới
- ☐ Xây dựng stratified split cho một biến quan trọng tự chọn
- ☐ Tạo feature mới và kiểm tra correlation cải thiện như thế nào
- ☐ Xây dựng ColumnTransformer pipeline với cả numerical và categorical features
- ☐ Giải thích train RMSE vs CV RMSE — phát hiện overfitting/underfitting
- ☐ Chạy GridSearchCV và đọc được `best_params_`, `best_score_`, `feature_importances_`
- ☐ Evaluate đúng chuẩn trên test set — chỉ một lần
- ☐ Save và load model hoàn chỉnh bằng joblib
- ☐ Giải thích 8 bước ML workflow cho người khác nghe được

Điểm tự đánh giá (0–10, dựa vào số kỹ năng ở trên đã hoàn thành): _____

1–2 điều tôi muốn tìm hiểu thêm sau bài học này:


